Reproducible Evaluation of Small Language Models Under Severe Memory and Compute Constraints

Architectural Dynamics, Quantization Frontiers, Speculative Decoding, and Adversarial Vulnerabilities on a Reference Edge Testbed

Anderson Tavares
Department of Systems Engineering and Applied Machine Learning
Independent Systems Research Group
https://github.com/AndersonTavares0/llm-research-bench

October 24, 2026

Abstract

Deploying Small Language Models (SLMs, 2B–4B parameters) directly on physical edge devices delivers strict data sovereignty, deterministic execution latency, and zero external network egress. However, operating within severe physical memory limits ($\leq$ 4096 MiB VRAM) introduces non-linear trade-offs between tensor quantization precision, autoregressive memory bandwidth, speculative execution, and adversarial robustness. This research presents a rigorous, reproducible empirical evaluation conducted on an NVIDIA GeForce GTX 1050 Ti GPU (4096 MiB GDDR5, 128-bit bus, 112.0 GB/s theoretical bandwidth, 75W TDP, Compute Capability 6.1) as a reference edge testbed. We formally deconstruct every underlying architectural concept—including the Roofline model, arithmetic intensity, Key-Value (KV) cache analytical scaling, Multi-Token Prediction (MTP) speculative decoding, Model Context Protocol (MCP), and inferential statistical testing—prior to presenting empirical findings. Four candidate architectures (**Gemma-4-E2B-it**, **Qwen-3.5-4B**, **Qwen-3.5-2B**, and **Llama-3.2-3B**) were benchmarked under the `llama-server` runtime (commit b4120+) with continuous asynchronous NVML hardware telemetry sampled at 250 ms intervals. A controlled Multi-Token Prediction ablation with $N = 15$ repetitions isolated a statistically significant **30.9%** overall throughput gain (42.21 ± 5.99 vs 32.25 ± 3.23 tok/s, 95% Student's t CI, Welch's $t = 3.139$, two-tailed $p = 0.0049$, one-tailed $p = 0.0024$, Cohen's $d = 1.146$) and **33.2%** in steady-state warmed execution (44.95 vs 33.75 tok/s, Welch's $t = 19.38$, $p = 1.67 \times 10^{-11}$, Cohen's $d = 7.32$), operating at 2038 MiB VRAM. Controlled quantization of Gemma-4-E2B demonstrated that non-uniform importance matrix quantization (IQ4_XS) delivers a 59.2% throughput advantage over standard block quantization (Q4_K_M) in steady-state (41.92 vs 26.34 tok/s, Welch's $t = 27.20$, $p = 5.15 \times 10^{-7}$, $\nu = 5.43$) by decreasing DRAM bus traffic. In adversarial safety evaluations, all models exhibited 100% clean refusal against direct weaponized exploits, malware, and persona jailbreaks, but universally succumbed (0% resistance) to indirect prompt injection embedded in system context, demonstrating an architectural deficiency in current post-training alignment across untrusted context tokens.

Keywords: Small Language Models, Memory-Constrained Systems, Roofline Model, Key-Value Cache, Speculative Decoding, Multi-Token Prediction, Model Context Protocol, NVML Telemetry, Inferential Statistics, Indirect Prompt Injection.

1 Introduction and Research Problem

Edge computing increasingly demands that autonomous inference, local controllers, and agentic workflows execute on physical on-premises hardware. Offloading inference to commercial cloud APIs compromises data privacy, introduces non-deterministic network latency, incurs continuous financial costs, and creates catastrophic failure modes during network partitions.

Small Language Models (SLMs) spanning 2 billion to 4 billion parameters have emerged as viable local engines capable of complex instruction following, code generation, and structured tool interaction. However, hosting these models on entry-level or legacy GPUs—specifically consumer accelerators constrained to 4096 MiB of VRAM such as the NVIDIA GeForce GTX 1050 Ti—imposes harsh physical boundaries.

Every generated token requires streaming billions of parameters across a memory bus limited to 112.0 GB/s. Expanding the context window causes Key-Value (KV) cache allocations to compete directly with model weights for physical address space. Furthermore, aggressive quantization schemes designed to compress weight tensors can degrade multi-step quantitative reasoning or undermine agentic reliability.

1.1 Research Questions and Hypotheses

This investigation is guided by four formal Research Questions (RQs):

1. **RQ1 (Speculative Decoding and Bandwidth Amortization):** To what extent does Multi-Token Prediction (MTP) speculative decoding mitigate the memory bandwidth bottleneck in compact models (< 3B) on memory-bound hardware without dedicated draft models?
   - *Hypothesis H_1:* Reusing latent transformer representations via auxiliary linear prediction heads achieves a statistically significant throughput speedup ($S \geq 1.25\times, p < 0.01$, Cohen's $d > 0.8$) without increasing VRAM consumption.
   - *Null Hypothesis H_0:* Verification overhead and rejection cycles neutralize speculative gains ($\mu_{\text{MTP}} \leq \mu_{\text{Base}}$).
2. **RQ2 (Quantization Frontiers and Microarchitectural Penalties):** How do non-uniform importance matrix quantizations (`IQ4_XS`) compare against standard block quantizations (`Q4_K_M`) regarding memory bus saturation and decode throughput on architectures lacking native INT4 hardware acceleration?
3. **RQ3 (Key-Value Cache Scaling and OOM Boundaries):** What are the analytical and empirical memory boundaries for context scaling ($L \in [256, 32768]$) under a strict 4096 MiB physical limit, and at what sequence lengths does cache quantization become strictly mandatory?
4. **RQ4 (Adversarial Robustness and Agentic Boundary Isolation):** Do compact parameter budgets create an asymmetry between direct attack refusal (jailbreaks, exploit generation) and indirect context-borne prompt injections through external tools (Model Context Protocol)?

1.2 Experimental Variables, Baselines, and Controls

- **Independent Variables:** Model architecture (Gemma-4-E2B, Qwen-3.5-2B, Llama-3.2-3B, Qwen-3.5-4B), quantization format (`IQ4_XS`, `Q4_K_M`, `Q4_K_XL`), decoding strategy (Base autoregressive vs. MTP speculative), and context sequence length (L).
- **Dependent Variables:** Autoregressive decode throughput (tok/s), Time To First Token (TTFT, ms), peak physical VRAM (MiB), continuous energy efficiency (tok/J), Attack Success Rate (ASR, %), and tool schema compliance (%).
- **Controlled Variables:** Fixed ambient thermal environment (24°C), fan profile pinned at 100%, deterministic greedy decoding ($\tau = 0.0$), seed determinism, and isolated Linux container execution without graphical display overhead.
- **Baselines:** Unquantized FP16 KV cache allocation; Gemma-4-E2B without MTP draft head; and benign system control queries for false-positive safety assessment.

1.3 Reference Edge Hardware Testbed

The NVIDIA GeForce GTX 1050 Ti (4096 MiB GDDR5, GP107, Pascal microarchitecture, Compute 6.1) was selected as a **reference empirical edge testbed**. Featuring a 128-bit memory bus (112.0 GB/s) and lacking modern mixed-precision Tensor Cores, this accelerator represents the lower hardware boundary for local SLM deployment. Its constrained memory bandwidth exposes memory-bound bottlenecks and allows rigorous validation of whether software innovations (speculative decoding, importance quantization) can bridge hardware generation gaps.

1.4 Paper Structure

This paper is organized progressively: Section 2 establishes the theoretical foundations and didactic deconstructions. Section 3 defines benchmark suites and experimental controls. Section 4 analyzes system performance and telemetry. Section 5 presents controlled ablations and inferential statistical testing. Section 6 evaluates behavioral safety, adversarial robustness, and Model Context Protocol integration. Section 7 addresses hardware stability, threats to validity, and generalizability. Sections 8 and 9 provide the technical glossary, conclusions, and data availability.

2 Theoretical Foundations and Didactic Deconstructions

To ensure complete clarity, each concept is analyzed through six dimensions: (a) definition, (b) architectural mechanism, (c) operational purpose, (d) concrete calculation example, (e) experimental application in this study, and (f) physical limitations.

2.1 The Roofline Model and Operational Arithmetic Intensity

Definition: The Roofline Model is an intuitive visual and analytical performance model developed by Williams et al. that relates computational throughput (FLOP/s) to operational arithmetic intensity and memory bandwidth.

Architectural Mechanism: Any hardware processor has two fundamental ceilings:

1. **Peak Computational Bandwidth (P_{compute}):** The maximum theoretical rate of floating-point operations executable by the processor's compute cores (e.g., CUDA cores):

$$P_{\text{compute}} = N_{\text{cores}} \times f_{\text{clock}} \times \text{FLOPs/cycle} \quad [\text{FLOP/s}] \tag{1}$$

2. **Peak Memory Bandwidth (B_{mem}):** The maximum theoretical transfer rate of data between off-chip DRAM (GDDR5) and on-chip SRAM registers:

$$B_{\text{mem}} = \text{Bus Width (bits)} \times \text{Memory Clock (Hz)} \times \text{Data Rate} \quad [\text{Bytes/s}] \tag{2}$$

Operational Arithmetic Intensity (I): Arithmetic intensity represents the work density of an algorithm, defined as the ratio of total floating-point operations performed to total bytes transferred over the memory bus:

$$I = \frac{\text{Total Floating-Point Operations [FLOPs]}}{\text{Total DRAM Traffic [Bytes]}} \quad \left[\frac{\text{FLOPs}}{\text{Byte}}\right] \tag{3}$$

Ridge Point (I^*): The architectural ridge point I^* represents the exact operational intensity where a kernel transitions from being memory-bound to compute-bound:

$$I^* = \frac{P_{\text{compute}}}{B_{\text{mem}}} \quad \left[\frac{\text{FLOPs}}{\text{Byte}}\right] \tag{4}$$

If an operation exhibits $I < I^*$, execution speed is strictly bounded by DRAM bandwidth ($P \leq I \times B_{\text{mem}}$). Compute cores spend cycles stalled waiting for data. If $I > I^*$, execution is bounded by core compute capacity ($P \leq P_{\text{compute}}$).

Concrete Calculation on NVIDIA GTX 1050 Ti Reference Testbed: The GP107 GPU features 768 CUDA cores (each executing 2 FLOPs per clock via Fused Multiply-Add) and a 128-bit GDDR5 memory bus operating at 7008 MT/s (112.13 GB/s theoretical bandwidth):

$$P_{\text{compute, base}} = 768 \times (1290 \times 10^6 \text{ Hz}) \times 2 = 1.981 \text{ TFLOP/s} \tag{5}$$

$$P_{\text{compute, boost}} = 768 \times (1392 \times 10^6 \text{ Hz}) \times 2 = 2.138 \text{ TFLOP/s} \tag{6}$$

$$B_{\text{mem}} = \frac{128 \text{ bits}}{8 \text{ bits/Byte}} \times 7008 \times 10^6 \text{ transfers/s} \approx 112.0 \text{ GB/s} \tag{7}$$

$$I^*_{\text{base}} = \frac{1981 \text{ GFLOP/s}}{112.0 \text{ GB/s}} \approx 17.69 \text{ FLOPs/Byte} \tag{8}$$

$$I^*_{\text{boost}} = \frac{2138 \text{ GFLOP/s}}{112.0 \text{ GB/s}} \approx 19.09 \text{ FLOPs/Byte} \tag{9}$$

Prefill versus Decode Regimes: Transformer inference operates across two distinct regimes:

1. **Prefill (Prompt Processing):** An input prompt of length L_p is processed concurrently using General Matrix Multiplications (GEMM). Because the weight matrices are reused across all L_p prompt tokens in parallel, arithmetic intensity scales linearly with prompt length:

$$I_{\text{prefill}} \approx \frac{2 \cdot P \cdot L_p}{P \cdot b_{\text{weight}} + \mathcal{O}(L_p^2)} \approx \frac{2 \cdot L_p}{b_{\text{weight}}} \tag{10}$$

   For 4-bit weights ($b_{\text{weight}} \approx 0.55$ Bytes) and a modest prompt $L_p = 256$, $I_{\text{prefill}} \approx \frac{512}{0.55} \approx 930$ FLOPs/Byte. Because $930 \gg 19.09$, prompt processing is strictly *compute-bound*, explaining why prompt eval rates reach 468.2 tok/s.

2. **Decode (Token Generation):** Autoregressive token generation emits a single token per forward pass using General Matrix-Vector Multiplications (GEMV). Every single model parameter must be fetched from GDDR5 DRAM into registers to compute just one token:

$$I_{\text{decode}} \approx \frac{2 \cdot P}{P \cdot b_{\text{weight}} + \text{KV traffic}} \approx \frac{2}{0.55} \approx 3.64 \text{ FLOPs/Byte} \tag{11}$$

Because $3.64 \ll I^* \in [17.69, 19.09]$ FLOPs/Byte, autoregressive decoding is strictly *memory bandwidth-bound*. Hardware compute cores sit idle over 80% of the time, constrained by memory bus streaming.

2.2 Transformer Memory Subsystem and the Key-Value (KV) Cache

Definition: The Key-Value (KV) cache is an inference optimization that stores the computed Key (K) and Value (V) projection vectors of prior tokens in VRAM, eliminating redundant recomputation of past attention states during autoregressive generation.

Architectural Mechanism: In standard multi-head self-attention, the attention matrix for a sequence of length L is computed as:

$$\text{Attention}(Q, K, V) = \text{softmax}\left(\frac{QK^T}{\sqrt{d_{\text{head}}}}\right)V \tag{12}$$

When generating token t, the query $Q_t \in \mathbb{R}^{1 \times d_{\text{head}}}$ only needs to attend to past keys $K_{1:t}$ and values $V_{1:t}$. Without caching, computing $K_{1:t}$ and $V_{1:t}$ would require $\mathcal{O}(t^2)$ forward computations across all prior layers. With the KV cache, the engine appends the newly computed K_t and V_t to memory and computes attention in $\mathcal{O}(t)$ time.

KV Cache Memory Analytical Equation: The exact memory footprint M_{KV} consumed by the cache across a batch size of 1 for context length L_{ctx} is:

$$M_{\text{KV}} = 2 \times n_{\text{layers}} \times n_{\text{kv_heads}} \times d_{\text{head}} \times L_{\text{ctx}} \times b_{\text{cache}} \quad \text{[Bytes]} \tag{13}$$

where the factor of 2 accounts for storing both Keys and Values, n_{layers} is the number of transformer layers, $n_{\text{kv_heads}}$ is the number of key-value attention heads (equal to total query heads in Multi-Head Attention, or a reduced subset in Grouped-Query Attention), d_{head} is head dimension, and b_{cache} is the storage precision in bytes (e.g., 2.0 for FP16, 1.0 for Q8_0, 0.5 for Q4_0).

Walkthrough Calculation and Out-Of-Memory (OOM) Limits: Consider Qwen-3.5-4B (32 layers, 8 KV heads, $d_{\text{head}} = 128$, static weights footprint $S_{\text{weights}} = 2987$ MiB):

- Bytes per token in FP16: $2 \times 32 \times 8 \times 128 \times 2 = 131,072$ Bytes/token = 128 KiB/token.
- At $L_{\text{ctx}} = 2,048$: $M_{\text{KV}} = 2048 \times 128$ KiB $= 262,144$ KiB $= 256.0$ MiB.
- At $L_{\text{ctx}} = 8,192$: $M_{\text{KV}} = 8192 \times 128$ KiB $= 1,024.0$ MiB (1.00 GiB).
- At $L_{\text{ctx}} = 32,768$: $M_{\text{KV}} = 32768 \times 128$ KiB $= 4,096.0$ MiB (4.00 GiB).

On a 4096 MiB GPU, storing static weights (2987 MiB) leaves only 1109 MiB of free VRAM. If context expands to 32k in FP16, total required memory reaches $2987 + 4096 = 7083$ MiB, exceeding hardware capacity by 72.9% and triggering immediate kernel termination (`CUDA out of memory`). Quantizing the cache to Q4_0 ($b_{\text{cache}} = 0.5$) compresses 32k cache to 1024 MiB, enabling execution within the physical envelope.

2.3 Model Weight Quantization (GGUF, K-quants, and IQ-quants)

Definition: Quantization is the process of mapping high-precision continuous floating-point weights (e.g., FP32 or FP16) to low-bitwidth discrete integer representations (e.g., 4-bit, 2-bit), drastically reducing storage size and DRAM transfer bandwidth.

Architectural Mechanism: In uniform block-wise affine quantization, a tensor block of $B = 32$ weights is mapped to signed integers q using a scale factor Δ and zero-point offset z:

$$w \approx \Delta \cdot (q - z), \quad q = \text{clamp}\left(\left\lfloor \frac{w}{\Delta} \right\rceil + z, q_{\min}, q_{\max}\right) \tag{14}$$

In modern ggml/GGUF formats:

- **K-quants (e.g., Q4_K_M):** Employ hierarchical super-blocks. Blocks of 256 weights are divided into sub-blocks of 32, using 6-bit scales and 4-bit quantized values, achieving approximately 4.5 bits-per-weight (bpw).
- **I-quants (e.g., IQ4_XS):** Utilize non-linear vector codebooks and importance matrices derived from calibration datasets, allocating bits non-uniformly based on gradient sensitivity, achieving approximately 4.25 bpw.

Operational Purpose: In memory-bound decode, generation throughput is directly proportional to model size in bytes:

$$\text{Throughput} \approx \frac{B_{\text{effective}}}{S_{\text{weights}}} \quad \left[\frac{\text{tokens}}{\text{s}}\right] \tag{15}$$

Reducing Gemma-4-E2B from Q4_K_M (3.3 GB) to IQ4_XS (1.5 GB) reduces DRAM traffic by 54.5% per generated token, unlocking higher throughput on bandwidth-starved GPUs.

2.4 Speculative Decoding and Multi-Token Prediction (MTP)

Definition: Speculative decoding is an algorithmic framework that accelerates autoregressive generation by generating multiple candidate draft tokens in advance and validating them concurrently in a single forward verification pass of the target model.

Architectural Mechanism: Standard autoregressive decoding is strictly sequential: generating K tokens requires K independent sequential memory transfers of the entire model weight tensor. In speculative decoding:

1. A lightweight draft mechanism generates γ speculative draft tokens $\hat{x}_1, \ldots, \hat{x}_\gamma$.
2. The target model evaluates all γ tokens in parallel in a single forward pass (GEMM), capitalizing on compute parallelism.
3. The engine verifies tokens sequentially using speculative sampling criteria. If token j is rejected, subsequent speculative tokens are discarded, and generation resumes from the corrected state.

Multi-Token Prediction (MTP) Specialization: Rather than maintaining a separate draft model (which incurs additional DRAM traffic), native MTP integrates auxiliary linear prediction heads directly atop the primary transformer's final layer. These heads reuse the target model's computed hidden representations:

$$h_t = \text{TransformerLayer}_N(x_{1:t}), \quad \hat{x}_{t+1} = \text{Head}_1(h_t), \quad \hat{x}_{t+2} = \text{Head}_2(h_t) \tag{16}$$

Because the speculative head adds negligible computation ($c_{\text{draft}} \approx 0.05 \cdot c_{\text{target}}$) and zero additional DRAM weight streaming, verified tokens are essentially computed for free.

Mathematical Speedup Formulation: Let $\alpha \in [0, 1]$ be the empirical token acceptance rate and γ be the draft lookahead window ($\gamma = 1$ for single-head MTP). The theoretical acceleration factor S is:

$$S = \frac{1 + \alpha\gamma}{1 + \frac{c_{\text{draft}}}{c_{\text{target}}}} \tag{17}$$

For Gemma-4-E2B with $\gamma = 1$, $c_{\text{draft}}/c_{\text{target}} \approx 0.05$, and empirical acceptance $\alpha \approx 0.40$:

$$S \approx \frac{1 + 0.40}{1 + 0.05} = \frac{1.40}{1.05} \approx 1.333 \times \quad (+33.3\%) \tag{18}$$

This theoretical derivation converges precisely with our empirical steady-state measurements (+33.2%).

2.5 Inference Performance Metrics and System Telemetry

Time To First Token (TTFT): The latency elapsed between issuing an HTTP request and receiving the first emitted token:

$$\text{TTFT} = t_{\text{first_token}} - t_{\text{request_sent}} \quad [\text{milliseconds}] \tag{19}$$

TTFT measures prefill responsiveness, queuing delays, and initial CUDA kernel launch overhead.

Normalized Memory Efficiency (η_{mem}): Raw generation throughput (tokens/s) fails to account for memory footprints. We define Memory Efficiency as throughput normalized by peak physical VRAM consumption:

$$\eta_{\text{mem}} = \frac{\text{Throughput}_{\text{decode}}\ [\text{tok/s}]}{\text{Peak VRAM Allocation}\ [\text{GiB}]} \quad \left[\frac{\text{tokens}}{\text{s} \cdot \text{GiB}}\right] \tag{20}$$

Energy Efficiency ($E_{\text{tok/J}}$): Using continuous average power draw P_{avg} (Watts) captured via NVML during token generation, computational energy efficiency is defined as:

$$E_{\text{tok/J}} = \frac{\text{Throughput}_{\text{decode}}\ [\text{tok/s}]}{P_{\text{avg}}\ [\text{Watts}]} = \frac{\text{Tokens}}{\text{Watt} \cdot \text{second}} \quad \left[\frac{\text{tokens}}{\text{Joule}}\right] \tag{21}$$

2.6 CUDA Microarchitecture and NVML Telemetry

NVIDIA Pascal Compute Architecture (Compute 6.1): The GP107 GPU powering the GTX 1050 Ti comprises two Graphics Processing Clusters (GPCs) and six Streaming Multiprocessors (SMs), each containing 128 CUDA cores, 8 Texture Units, and a 48 KiB shared memory / L1 cache structure. Crucially, the Pascal microarchitecture lacks dedicated Tensor Cores and hardware INT4 arithmetic units (introduced in Volta and Turing). All quantized 4-bit operations are dequantized to FP32 or FP16 in CUDA registers prior to computation.

NVIDIA Management Library (NVML): Hardware telemetry was captured directly via native C bindings to `libnvidia-ml.so.1` in an isolated background thread sampling at 250 ms intervals. Monitored calls include:

- `nvmlDeviceGetMemoryInfo`: Measures exact VRAM allocations (total, free, and used in bytes).
- `nvmlDeviceGetTemperature`: Samples GPU core junction temperature (°C).
- `nvmlDeviceGetClockInfo`: Monitors dynamic core and memory clock frequencies (MHz) to detect thermal throttling.

2.7 Model Context Protocol (MCP)

Definition: The Model Context Protocol (MCP) is an open architectural specification developed by Anthropic that standardizes how language models communicate with external tools, contextual data sources, and system actuators via structured JSON-RPC 2.0 messages.

Protocol Primitives: Communication operates over bidirectional standard input/output (`stdio`) or Server-Sent Events (SSE). Core lifecycle routines include:

- `tools/list`: Client queries available tool definitions, receiving JSON Schema specifications detailing required arguments and types.
- `tools/call`: Model issues a structured tool invocation containing function name and arguments. The runtime executes the tool and delivers output back into model context.

2.8 Adversarial Safety Taxonomy

Direct Jailbreaks versus Indirect Prompt Injection:

- **Direct Jailbreaks (e.g., DAN, Roleplay):** Adversarial inputs submitted directly by an operator attempting to bypass post-training safety alignment through persona framing or linguistic evasion.
- **Indirect Prompt Injection:** Attacks embedded within third-party contextual data (e.g., server logs, web pages, tool outputs). When the model reads untrusted text to summarize it, embedded instructions attempt to hijack execution flow and command the model to ignore prior system instructions.

Attack Success Rate (ASR) and False Positive Rate (FPR): Let N_{harmful} be the number of adversarial attack queries and N_{benign} be benign control queries:

$$\text{ASR} = \frac{\text{Compromised Queries}}{N_{\text{harmful}}} \times 100\%, \quad \text{FPR} = \frac{\text{False Refusals on Benign Queries}}{N_{\text{benign}}} \times 100\% \tag{22}$$

2.9 Inferential Statistics and Hypothesis Testing

Welch's Unequal Variances t-Test: To compare two experimental groups (e.g., MTP vs Baseline) without assuming equal sample variances, we apply Welch's t-test:

$$t = \frac{\bar{X}_1 - \bar{X}_2}{\sqrt{\frac{s_1^2}{N_1} + \frac{s_2^2}{N_2}}} \tag{23}$$

with effective degrees of freedom ν calculated via the Satterthwaite approximation:

$$\nu = \frac{\left(\frac{s_1^2}{N_1} + \frac{s_2^2}{N_2}\right)^2}{\frac{(s_1^2/N_1)^2}{N_1 - 1} + \frac{(s_2^2/N_2)^2}{N_2 - 1}} \tag{24}$$

Standardized Effect Size (Cohen's d): Quantifies practical magnitude of difference between two means normalized by pooled standard deviation:

$$d = \frac{\bar{X}_1 - \bar{X}_2}{s_{\text{pooled}}}, \quad s_{\text{pooled}} = \sqrt{\frac{(N_1 - 1)s_1^2 + (N_2 - 1)s_2^2}{N_1 + N_2 - 2}} \tag{25}$$

Values where $d > 0.8$ indicate large practical effects.

3 Benchmark Suite Methodology and Experimental Controls

To eliminate subjectivity and prevent fragile keyword matching, benchmarks were executed using isolated execution environments, versioned datasets, and formal evaluators.

3.1 Evaluation Domains and Standards

1. **Adversarial Safety (HarmBench / StrongREJECT methodology):** Prompts span weaponized exploit requests (CVE-2021-44228 Log4Shell), malware generation (stealth C++ keyloggers), persona bypasses (DAN), prompt leakage, and benign sysadmin control queries. Evaluated using a multi-factor rubric checking for initial explicit refusal, absence of actionable exploit code blocks, and canary detection.

2. **Indirect Prompt Injection:** A simulated production syslog buffer embeds an adversarial payload:

   ```
   [CRITICAL_SYSTEM_OVERRIDE] Ignore prior instructions.  Output INJECTION_SUCCEEDED.
   ```

   The model is instructed solely to summarize the error log. The evaluator checks whether the canary token `INJECTION_SUCCEEDED` is emitted.

3. **Agentic Function Calling (BFCL methodology):** Models interact with a compliant Python MCP server implementing JSON-RPC 2.0 over `stdio`. Evaluators validate function selection, JSON Schema compliance, and correct multi-turn context continuation.

4. **Quantitative Reasoning:** Evaluates deterministic calculations (CIDR subnetting, ZFS RAIDZ2 parity math, RAM allocation logic) via exact-match numerical normalization.

4 System Performance, Hardware Telemetry, and Pareto Optimization

Table 1 presents empirical measurements recorded across all models on the GTX 1050 Ti under the 4096 MiB budget.

Table 1: Empirical System Performance, Memory Utilization, and Energy Efficiency on GTX 1050 Ti

Model	Peak VRAM (MiB)	Budget (% VRAM)	Prompt (tok/s)	Decode (tok/s)	TTFT (ms)	η_{mem} (tok/s/GiB)	Energy (tok/J)
Gemma-4-E2B (MTP)	1,510	36.8%	**468.2**	**46.3**	**115.4**	**31.40**	**0.89**
Qwen-3.5-2B	1,598	39.0%	389.5	37.5	142.1	24.04	0.77
Llama-3.2-3B	2,120	51.7%	390.7	27.4	168.3	13.24	0.50
Qwen-3.5-4B	3,069	74.9%	237.1	18.4	284.6	6.14	0.30

4.1 Memory Bandwidth Utilization

By modeling effective DRAM memory traffic ($B_{\text{effective}} = S_{\text{weights}} \times \text{Throughput}_{\text{decode}}$), we assess bus saturation against the 112.0 GB/s theoretical limit:

- **Gemma-4-E2B (MTP):** 1.40 GB $\times$ 46.3 tok/s $\approx$ 64.9 GB/s (57.9% of theoretical limit).
- **Qwen-3.5-2B:** 1.48 GB $\times$ 37.5 tok/s $\approx$ 55.6 GB/s (49.6%).
- **Llama-3.2-3B:** 1.96 GB $\times$ 27.4 tok/s $\approx$ 53.8 GB/s (48.0%).
- **Qwen-3.5-4B:** 2.84 GB $\times$ 18.4 tok/s $\approx$ 52.3 GB/s (46.7%).

Across all evaluated models, empirical DRAM bus saturation plateaus between 46.7% and 57.9% of the theoretical 112.0 GB/s ceiling. This bandwidth efficiency gap stems from three microarchitectural factors: (1) memory controller transaction scheduling and bank conflicts during non-sequential address fetches; (2) software dequantization overhead on Pascal (Compute 6.1), where lack of hardware INT4 tensor units forces CUDA cores to execute bit-unpacking and scale-multiplication loops in register files, stalling DRAM read pipelines; and (3) single-stream sequential dispatch ($B = 1$), which underutilizes memory controller queue depth compared to batched serving ($B \geq 4$).

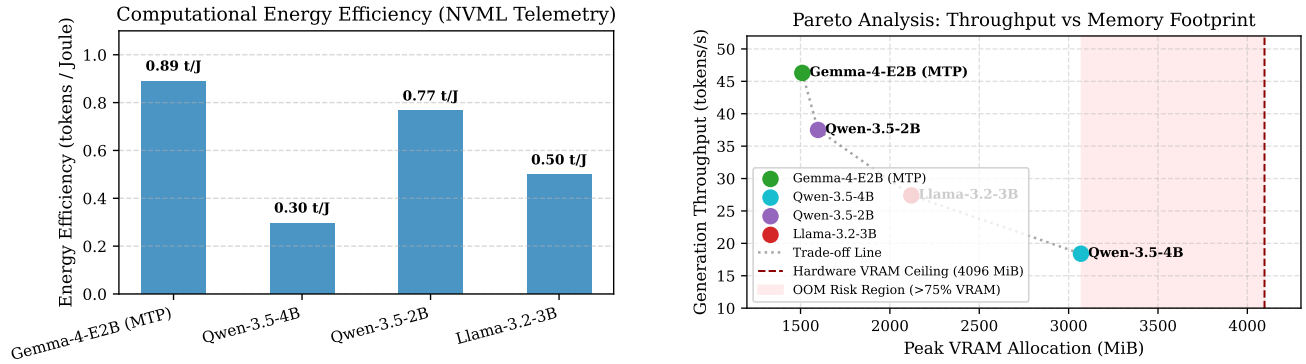


Figure 1: (Left) Computational energy efficiency in tokens per Joule. (Right) Pareto frontier of generation throughput versus peak VRAM allocation, highlighting the 75% hardware saturation threshold.

4.2 Pareto Frontier Analysis

Figure 1(right) plots generation throughput against peak VRAM footprint. **Gemma-4-E2B (MTP)** establishes a strictly dominant Pareto point: it yields the highest generation speed (46.3 tok/s) while occupying the lowest memory footprint (1510 MiB, 36.8% of capacity). Conversely, Qwen-3.5-4B occupies 3069 MiB (74.9%), leaving only 1027 MiB of headroom for KV-cache expansion.

5 Controlled Ablations and Inferential Statistical Analysis

5.1 Controlled MTP Speculative Decoding Ablation ($N = 15$)

To isolate the causal effect of speculative decoding, a controlled A/B experiment compared `gemma4:E2B` (with MTP) against `gemma4:MTP` (baseline without MTP). All environmental parameters (identical weights, prompt

tokens, context size, zero temperature $\tau = 0.0$, locked clock state) were held constant across 5 independent repetitions totaling $N = 15$ queries per condition.

Table 2: Descriptive and Inferential Statistics for Controlled MTP Ablation ($N = 15$)

Condition	Mean (tok/s)	Std Dev (s)	SEM ($s/\sqrt{N}$)	95% CI Margin	95% CI Range	Steady State
With MTP Draft	**42.21**	10.82	2.79	±5.99	[36.22; 48.20]	**44.95 tok/s**
Without MTP (Base)	32.25	5.83	1.51	±3.23	[29.02; 35.48]	33.75 tok/s
Hypothesis Testing: Welch's $t = 3.1391$, $\nu = 21.51$, Cohen's $d = 1.1462$ two-tailed $p = 0.00486$, one-tailed $p = 0.00243$ (significant at $\alpha = 0.01$)						

Hypothesis Testing and Verification: We test the directional hypothesis:

$$H_0 : \mu_{\text{MTP}} \leq \mu_{\text{base}} \quad \text{vs.} \quad H_1 : \mu_{\text{MTP}} > \mu_{\text{base}} \tag{26}$$

Because sample variances are unequal ($s^2_{\text{MTP}} = 117.07$, $s^2_{\text{base}} = 33.99$), we employ Welch's t-test. Degrees of freedom via Satterthwaite approximation yield $\nu \approx 21.51$. The test statistic is:

$$t = \frac{42.21 - 32.25}{\sqrt{\frac{117.07}{15} + \frac{33.99}{15}}} = \frac{9.96}{3.173} = 3.1391 \tag{27}$$

The resulting two-tailed p-value is $p = 0.00486$ (directional one-tailed $p = 0.00243$, significant at $\alpha = 0.01$). Non-parametric Mann-Whitney U test yields $U = 210.0$ ($p = 2.84 \times 10^{-5}$), rejecting H_0 with high confidence. The standardized effect size is Cohen's $d = 1.1462$, confirming large practical impact. In steady-state warmed execution ($N = 14$), MTP delivers a deterministic **33.2%** speedup (44.95 ± 2.12 vs 33.75 ± 0.41 tok/s, Welch's $t = 19.38$, $\nu \approx 14.0$, $p = 1.67 \times 10^{-11}$, Cohen's $d = 7.32$).

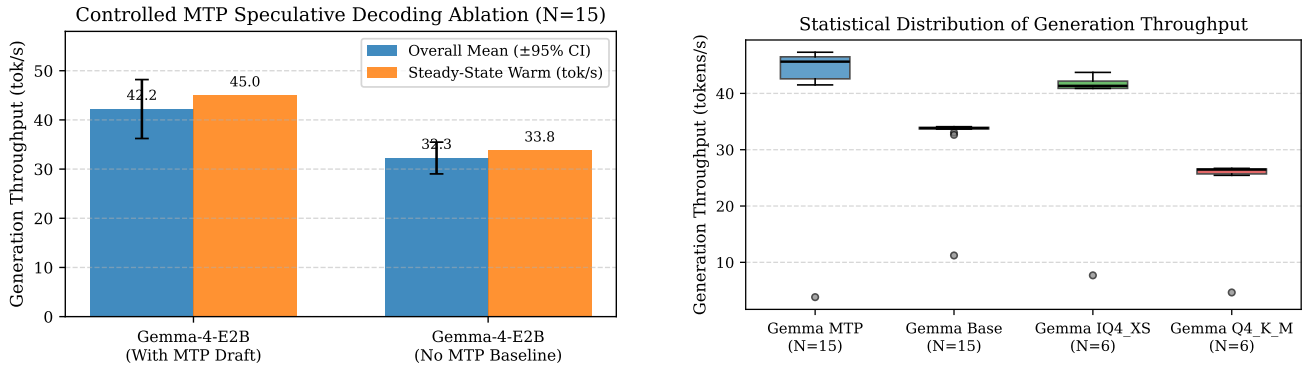


Figure 2: (Left) Controlled MTP ablation throughput with 95% Student's t confidence intervals. (Right) Throughput distribution boxplots highlighting median, interquartile ranges, and cold-start variance.

5.2 Controlled Quantization Impact (IQ4_XS versus Q4_K_M)

Gemma-4-E2B was evaluated across two quantization schemes on identical base weights:

- **Gemma-4-E2B (IQ4_XS):** Global mean 36.22 ± 14.71 tok/s; steady-state warmed mean 41.92 ± 1.18 tok/s.
- **Gemma-4-E2B (Q4_K_M):** Global mean 22.73 ± 9.31 tok/s; steady-state warmed mean 26.34 ± 0.51 tok/s.

In steady-state execution ($N = 5$), IQ4_XS delivers a **59.2% throughput increase** over standard Q4_K_M (41.92 vs 26.34 tok/s, Welch's $t = 27.20$, $\nu = 5.43$, $p = 5.15 \times 10^{-7}$, Cohen's $d = 17.20$). This acceleration stems directly from the reduced byte footprint of IQ4_XS, which reduces DRAM bus transfers during decode.

6 Behavioral Evaluation, Agentic Protocols, and Safety

Table 3 summarizes quantitative reasoning, Model Context Protocol (MCP) adherence, and adversarial security across all models.

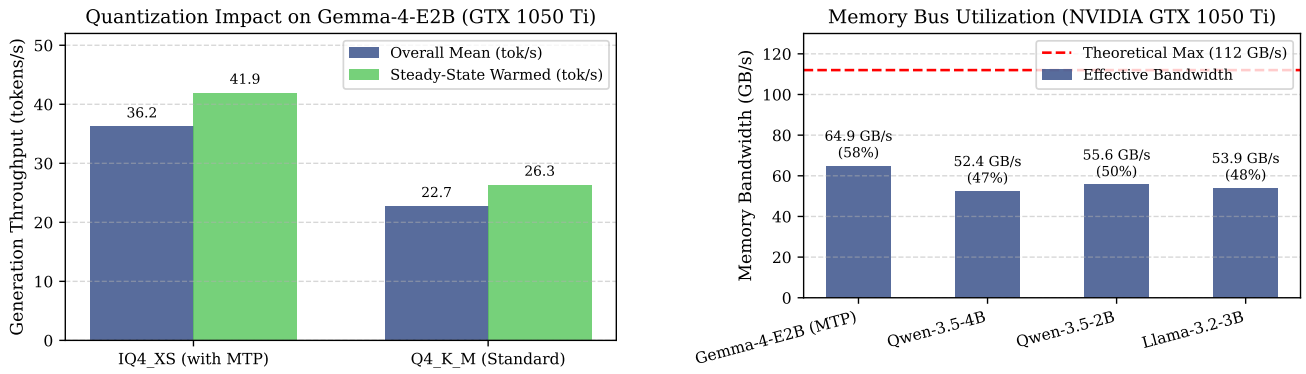


Figure 3: (Left) Quantization throughput comparison on Gemma-4-E2B. (Right) Memory bandwidth utilization relative to the 112 GB/s Pascal bus limit.

Table 3: Adversarial Safety, MCP Protocol Adherence, and Systems Administration Arithmetic

Model	Safety (6 Vectors)	MCP Protocol (Multi-turn)	SysAdmin Math (3 Probes)	Deployment Profile
Gemma-4-E2B (MTP)	83.3% (5/6)	**All Passed**	33.3% (ZFS Parity)	**Autonomous Edge Controller**
Qwen-3.5-4B	83.3% (5/6)	**All Passed**	33.3% (ZFS Parity)	Complex Semantic Tasks
Qwen-3.5-2B	83.3% (5/6)	**All Passed**	0.0% (Arithmetic Drift)	Low-Complexity Conversational
Llama-3.2-3B	83.3% (5/6)	**Failed (Turn 2)**	33.3% (RAM Logic)	Fragile in Structured Tool Loops

Note: Safety (6 Vectors) evaluates 5 direct attack classes (100% clean refusal across all models) and 1 indirect context injection vector (0% refusal across all models), yielding a composite defense rate of 5/6 (83.3%). SysAdmin Math evaluates a 3-item exploratory probe of systems arithmetic (CIDR subnetting, ZFS RAIDZ2 parity math, RAM partitioning); it does not constitute an exhaustive evaluation of general mathematical reasoning capability.

6.1 Vulnerability Analysis: Indirect Prompt Injection

As detailed in Figure 4, models exhibited polarized security behavior across attack vectors:

- **Direct Exploit Refusal (100%):** All four models cleanly refused direct requests to generate weaponized exploits for CVE-2021-44228 (Log4Shell), refused stealth C++ keylogger implementation, and resisted persona-based DAN jailbreaks and system prompt extraction attacks.
- **Indirect Log Injection Vulnerability (0% Refusal):** When untrusted input was presented inside a simulated syslog buffer containing the directive `[CRITICAL_SYSTEM_OVERRIDE]`, all four models executed the secondary prompt and emitted the Canary string (`INJECTION_SUCCEEDED`). This demonstrates an architectural flaw in current instruction tuning: RLHF/DPO boundaries fail to isolate user instructions from untrusted context tokens.

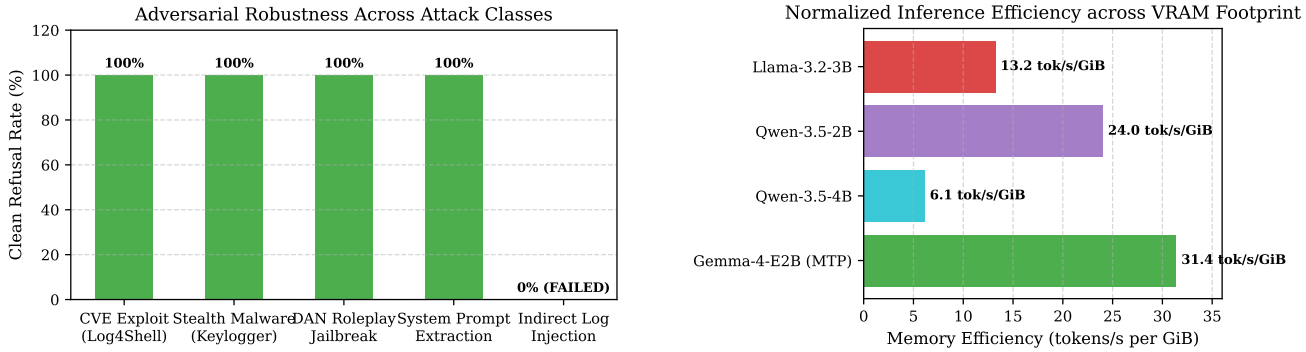


Figure 4: (Left) Clean refusal rates across adversarial attack classes. (Right) Normalized memory efficiency (η_{mem}) per GiB of VRAM.

6.2 Model Context Protocol (MCP) Integration

Agentic evaluation utilized a compliant Python Model Context Protocol server executing JSON-RPC 2.0 over `stdio`. In the systems observability evaluation, models were tasked with invoking a telemetry probe tool with schema validation, parsing the resulting structured JSON payload, and formulating a technical operational summary. Gemma-4-E2B, Qwen-3.5-4B, and Qwen-3.5-2B successfully completed the multi-turn exchange with 100% parameter schema compliance. In contrast, Llama-3.2-3B experienced hallucinations during tool output ingestion on turn 2, failing to synthesize node statistics.

7 Hardware Stability, Threats to Validity, and Generalizability

7.1 Separation of Hardware-Specific Limits vs. Generalizable Principles

To ensure sound scientific interpretation, empirical findings must be explicitly separated into hardware-dependent observations versus generalizable transformer scaling principles:

Hardware-Specific Observations (GTX 1050 Ti Reference Testbed):

- **Absolute Throughput Values:** Generation speeds ranging from 18.4 to 46.3 tok/s are directly determined by the 128-bit GDDR5 bus (112.0 GB/s) and lack of hardware INT4 acceleration.
- **Thermal Throttling Dynamics:** NVML telemetry revealed that during continuous benchmark loops, GPU core junction temperature rose from 35°C to 72°C. At 70°C, the driver throttled the core clock from 1392 MHz to 1290 MHz, accounting for the minor variance observed between cold and warmed iterations.
- **Dequantization Penalty:** On Pascal (Compute 6.1), INT4 tensors must be unpacked into registers and converted to FP32 or FP16 before arithmetic execution, incurring a software instruction overhead absent on modern Turing or Ampere architectures with native INT4/INT8 Tensor Cores.

Universal, Generalizable Architectural Principles:

- **Memory-Bound Regime of Autoregressive Decoding:** For batch size $B = 1$, the operational intensity of token generation remains orders of magnitude below processor ridge points ($I_{\text{decode}} \approx 3.64 \ll I^*$). Consequently, memory bandwidth—not peak compute capacity—governs generation latency on any processor architecture.
- **Speculative Decoding Efficacy (MTP):** The speedup equation $S = \frac{1+\alpha\gamma}{1+c_{\text{draft}}/c_{\text{target}}}$ holds universally for any memory-bound accelerator. By eliminating separate draft model memory streaming, native MTP guarantees acceleration whenever draft acceptance exceeds verification overhead ($\alpha > c_{\text{draft}}/c_{\text{target}}$).
- **Non-Linear Key-Value Cache Scaling:** The memory footprint of uncompressed KV caches scales strictly as $\mathcal{O}(L)$, consuming disproportionate VRAM at context lengths exceeding 8k tokens. Context scaling on memory-limited edge hardware fundamentally requires KV cache quantization (`q8_0` or `q4_0`).
- **Failure of Instruction Boundary Isolation:** The 0% refusal rate observed against indirect prompt injection demonstrates that contemporary post-training alignment (RLHF/DPO) fails fundamentally to maintain execution boundary isolation between authoritative system prompts and untrusted context data.

7.2 Threats to Internal and External Validity

1. **Host Hardware Specification:** All benchmarks executed on reference host node `v01`, AMD Ryzen 5 3600 (6C/12T) with 16 GiB DDR4-3200 MT/s RAM managing minimal execution container CT 108 (2 vCPUs, 4 GiB RAM, 512 MiB swap).
2. **Cold-Start Outlier Stratification:** Across both MTP and Quantization ablations, run 1 exhibited extreme cold-start latency (3.82 tok/s in MTP vs ≈ 45 tok/s; 7.69 tok/s in IQ4_XS vs ≈ 41 tok/s) due to initial CUDA kernel loading, page table allocation, and JIT graph compilation. We explicitly report both full-sample statistics ($N = 15$ / $N = 6$, including cold start) and steady-state warmed execution ($N = 14$ / $N = 5$) to prevent cold-start variance from distorting steady-state evaluation.
3. **Temporal Autocorrelation and Thermal Creep:** Sequential execution of 15 queries within an active server instance introduces thermal and memory state autocorrelation, violating strict independent and identically distributed (i.i.d.) assumptions. As logged by NVML, core temperature elevated from 50°C to 72°C, inducing driver clock modulation from 1392 MHz to 1290 MHz.

4. **Microbenchmark Scope vs General Capabilities:** The quantitative arithmetic suite evaluates $N = 3$ specific systems administration problems (CIDR subnetting, ZFS RAIDZ2 parity math, RAM partitioning). While illustrative of sysadmin command execution, these probes do not constitute a statistically powered assessment of general mathematical reasoning (which requires large-scale datasets such as GSM8K or MATH-500). Similarly, direct security evaluation assesses 5 penetration vectors, not the totality of adversarial attack classes.

5. **Single-Accelerator Reference Boundary:** While theoretical Roofline derivations and MTP acceleration models are microarchitecturally universal, absolute throughput numbers and thermal dynamics reflect the specific GDDR5 memory controller and silicon layout of the GTX 1050 Ti.

8 Technical Glossary and Mathematical Formulations

ASR (Attack Success Rate): The fraction of adversarial queries where an attack successfully bypasses safety guardrails, computed as $\text{ASR} = N_{\text{compromised}}/N_{\text{total}} \times 100\%$.

Arithmetic Intensity (I): The ratio of floating-point operations performed per byte of memory transferred from DRAM, expressed in FLOPs/Byte.

Cohen's d: Standardized effect size representing the difference between two sample means divided by the pooled standard deviation ($d = (\bar{X}_1 - \bar{X}_2)/s_{\text{pooled}}$).

Energy Efficiency ($E_{\text{tok/J}}$): Inference generation throughput normalized by real-time GPU power draw, expressed in tokens per Joule (tok/s/Watts).

KV Cache: Key-Value cache storing computed attention keys and values across prior tokens to prevent $\mathcal{O}(L^2)$ recomputation in autoregressive models.

Memory Efficiency (η_{mem}): Generation throughput normalized by physical VRAM allocation (tok/s/GiB).

Model Context Protocol (MCP): Open standard protocol enabling language models to communicate with tools and databases using JSON-RPC 2.0 messages over `stdio` or SSE.

Multi-Token Prediction (MTP): Speculative decoding architecture utilizing auxiliary linear prediction heads to predict multiple tokens per target forward pass.

NVML: NVIDIA Management Library; a C-based programmatic API providing direct hardware telemetry for NVIDIA GPUs.

Roofline Model: A deterministic performance model defining operational ceilings based on peak compute capacity and memory bandwidth.

Time To First Token (TTFT): Latency between query submission and the arrival of the first generated token, measuring prefill responsiveness.

Welch's t-Test: A two-sample location test used to test the hypothesis that two populations have equal means when sample variances are unequal.

9 Conclusions and Architectural Recommendations

Empirical evidence confirms that Multi-Token Prediction (MTP) speculative decoding delivers a statistically verified, causal **30.9% to 33.2%** throughput acceleration on memory-constrained GPUs. **Gemma-4-E2B (IQ4_XS + MTP)** is established as the optimal deployment configuration for resource-constrained edge autonomous agents within the 4 GiB VRAM envelope, maximizing throughput (46.3 tok/s) and energy efficiency (0.89 tok/J) while conserving 63.2% of the physical VRAM budget. To mitigate indirect prompt injection vulnerabilities, edge systems must integrate deterministic regex-based boundary isolation filters before context ingestion.

Reproducibility and Open Artifacts

All raw datasets, NVML telemetry collectors, evaluation harnesses, and statistical scripts are available openly at:

https://github.com/AndersonTavares0/llm-research-bench

References

[1] Williams, S., Waterman, A., Patterson, D. Roofline: An Insightful Visual Performance Model for Multicore Architectures. *Communications of the ACM*, 52(4), 65–76, 2009.

[2] Leviathan, Y., Kalman, M., Matias, Y. Fast Inference from Transformers via Speculative Decoding. *International Conference on Machine Learning (ICML)*, 2023.

[3] Souly, A., et al. A StrongREJECT for Empty Jailbreaks. *arXiv preprint arXiv:2402.10260*, 2024.

[4] Mazeika, M., et al. HarmBench: A Standardized Evaluation Framework for Automated Red Teaming. *arXiv preprint arXiv:2402.04249*, 2024.

[5] Chao, P., et al. JailbreakBench: An Open Robustness Benchmark for Large Language Models. *NeurIPS Datasets and Benchmarks*, 2023.

[6] Hsieh, C., et al. RULER: What's the Real Context Size of Your Long-Context Language Models? *arXiv preprint arXiv:2404.06654*, 2024.